

COMPILER CONSTRUCTION

Project Documentation

A Modular Expression Compiler: Lexical Analysis, Syntax Analysis,
and Intermediate Code Generation

PRESENTED BY:

- REHMAN NASIR (Leader)
- ABDUL RAFAIH
- ABDULLAH SHAH
- NOAMAN ADNAN
- AHSAN AHMAD
- SHEHRYAR AHMAD

Table of Contents

Table of Contents	2
1. Introduction	3
2. Compiler Phases Overview	4
3. Lexical Analysis (Tokenization)	4
4. Syntax Analysis (Parsing — Infix to Postfix)	4
5. Intermediate Code Generation (Three-Address Code)	5
5.1 Problem Statement	5
5.2 Worked Example	5
6. Testing and Validation	6
7. Implementation Flow	7
8. Software Development Life Cycle (SDLC)	7
9. Conclusion	9

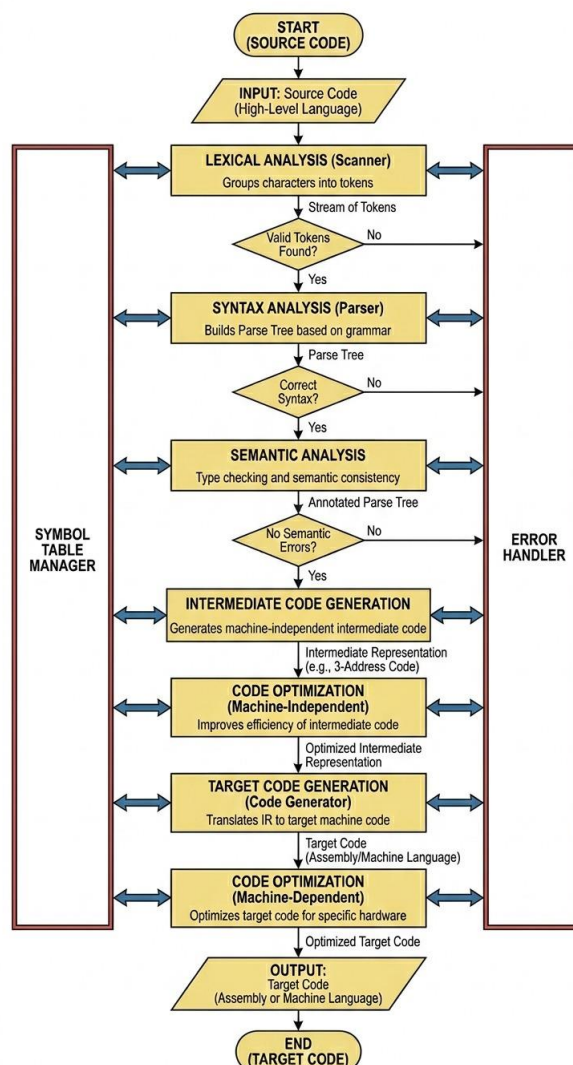
1. Introduction

This document describes the workflow of a simple compiler front end that reads an arithmetic expression written in infix notation and produces an equivalent sequence of intermediate three-address code (TAC) instructions. The compiler is organized into four distinct phases: **lexical analysis**, **syntax analysis**, **semantic checks**, and **intermediate-code generation**. Each phase transforms the source step by step, with a clean separation of concerns between stages.

In this project, the components map directly onto these phases:

- **Tokenizer** (lexical analyzer) — breaks the raw input string into a sequence of meaningful symbols, or tokens.
- **Parser** — converts the infix token stream into a structured postfix (Reverse Polish Notation) form.
- **Code Generator** — produces three-address code instructions from the postfix sequence.

The remainder of this document details each component in turn and outlines the Software Development Life Cycle (SDLC) stages followed throughout the project.



2. Compiler Phases Overview

A traditional compiler pipeline converts raw source text into tokens (lexical analysis), checks grammatical structure (syntax analysis), performs semantic analysis, generates an intermediate representation, and ultimately produces target code. The figure below illustrates the major phases of a compiler, from lexical analysis through to final code generation, along with the supporting symbol table and error-handling subsystems.

This modular design isolates concerns and simplifies downstream processing. For example, the **tokenize** function extracts identifiers, literals, operators, and parentheses as discrete tokens. The **infixToPostfix** function then applies a stack-based shunting-yard algorithm to reorder these tokens according to operator precedence. Finally, **postfixToTAC** emits three-address code instructions for each operation, using temporary variables (t1, t2, ...) to hold intermediate results.

By following this phased approach — lexing, parsing, semantic checks, and code generation — the compiler cleanly implements each transformation stage as an independent, testable unit.

3. Lexical Analysis (Tokenization)

The lexical analysis stage reads the input string character by character and groups characters into meaningful tokens. According to standard compiler theory, the lexical analyzer scans the source code and groups characters into tokens based on the rules of the language.

In this project, the tokenize function performs the following operations:

- **Skips whitespace:** any spaces in the input are ignored.
- **Builds multi-character tokens:** consecutive letters or digits are accumulated into identifiers or numeric literals (e.g., "abc", "123").
- **Separates operators and punctuation:** when a non-alphanumeric character (such as +, -, *, /, (, or)) is encountered, the current token (if any) is closed off, and the operator or parenthesis is added as its own separate token.

For example, the input `int x = 10;` would be tokenized as the sequence `["int", "x", "=", "10", ";"]`. In this project, testing the tokenizer with a range of inputs — including invalid characters — helps ensure robust and reliable token formation.

4. Syntax Analysis (Parsing — Infix to Postfix)

Once the input has been tokenized, the next phase checks grammatical structure. This project implements a parser by converting the infix token list into postfix (Reverse Polish) notation using a stack-based shunting-yard method, an algorithm originally devised by Edsger Dijkstra. The algorithm processes tokens in order according to the following rules:

1. **Operands** (numbers, variables) are immediately appended to the postfix output.
2. **Left parentheses** are pushed onto the operator stack to mark the start of a sub-expression.
3. **Right parentheses** cause operators to be popped to the output until the matching left parenthesis is found.

4. **Operators** (+, −, ×, ÷): while the operator at the top of the stack has higher or equal precedence, it is popped to the output; the current operator is then pushed onto the stack.

The result is a list of tokens in postfix order that correctly encodes the original infix expression along with its precedence. For example, the expression $3 + 4 \times (2 - 1)$ becomes $3\ 4\ 2\ 1\ -\ \times\ +$ in postfix form. The precedence function assigns higher values to multiplication and division than to addition and subtraction, enforcing the correct evaluation order, and mismatched parentheses are explicitly checked.

According to compiler theory, this parsing stage ensures that the source code follows the grammatical rules of the language and reports an error if it does not. In practice, unit tests for the parser feed in expressions and verify the resulting postfix output, confirming that operator precedence and parenthesization are handled correctly.

5. Intermediate Code Generation (Three-Address Code)

5.1 Problem Statement

After parsing, the next step is to generate Three-Address Code (TAC) from the postfix token sequence. TAC is an intermediate representation in which each instruction has at most three operands — two source operands and one destination. The implementation, **postfixToTAC**, maintains a stack of operand names. For each operator token encountered, it:

1. Pops the top two operands from the stack;
2. Creates a new temporary variable via `newTemp()`;
3. Outputs an instruction of the form `t = op1 <operator> op2`;
4. Pushes the new temporary back onto the stack for use in subsequent operations.

This matches the standard TAC pattern $x = y\ op\ z$. The use of temporary variables clarifies evaluation order and simplifies later optimization passes.

5.2 Worked Example

Consider the input expression $a + b * c$. The table below traces the expression through each stage of the pipeline.

Stage	Output
Input expression	$a + b * c$
Tokens	<code>["a", "+", "b", "*", "c"]</code>
Postfix	$a\ b\ c\ *\ +$
TAC output	$t1 = b * c$ $t2 = a + t1$

Each line follows the form *temp = operand1 operator operand2*. In this example, the multiplication is performed first ($t1 = b * c$), and the result is then added to a ($t2 = a + t1$). The final temporary, $t2$, holds the result of the overall expression. This approach provides a simplified intermediate layer that a back-end code generator could later translate into machine-specific instructions.

6. Testing and Validation

Comprehensive testing ensures each phase of the compiler works correctly in isolation and in combination. The general strategy is to write sample expressions and check the output produced at every stage of the pipeline.

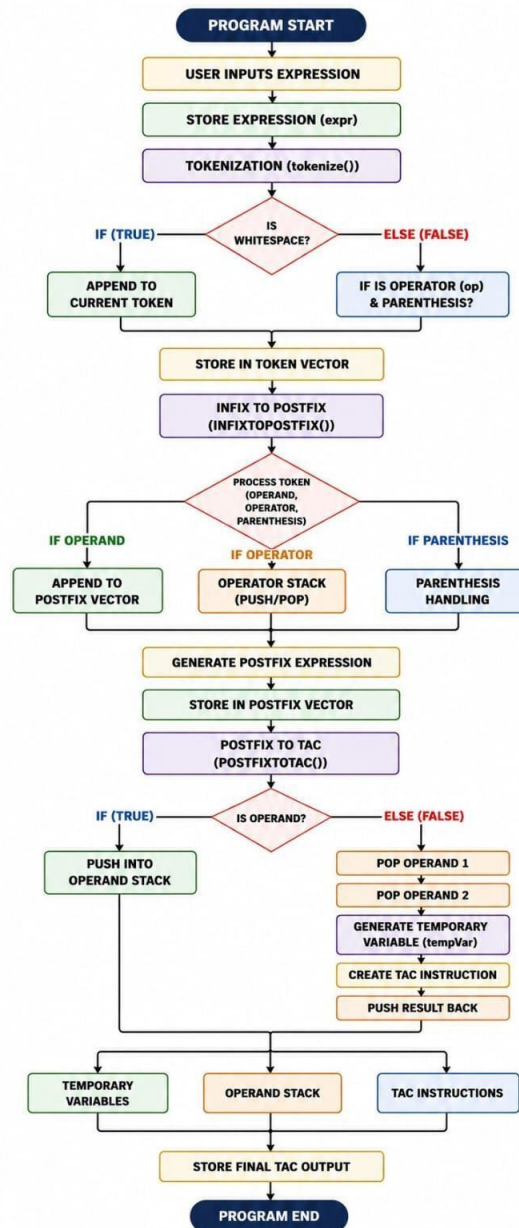
- **Tokenization tests:** input strings with varied spacing, multi-digit numbers, and invalid symbols should produce the expected token lists.
- **Parsing tests:** known infix expressions should yield the correct postfix sequence. Edge cases such as unbalanced parentheses or illegal operator sequences should trigger errors, since the syntax phase is designed to detect malformed input.
- **TAC generation tests:** given a postfix expression, the resulting sequence of TAC instructions should match the expected manual conversion. For example, for the postfix expression $a \ b \ + \ c \ *$, the generated TAC should be $t1 = a + b$ followed by $t2 = t1 * c$.

By automating these tests — for example, using a unit-testing framework — developers can quickly detect regressions, since any deviation indicates a bug in the corresponding phase. Compiler theory places strong emphasis on error handling: each phase should report meaningful errors, such as a “mismatched parenthesis” error during parsing. In practice, the test suite verifies that invalid input is handled gracefully and produces an appropriate error message rather than causing the program to crash.

7. Implementation Flow

The diagram below traces the detailed control flow of the implementation, from program start through tokenization, infix-to-postfix conversion, and postfix-to-TAC generation, down to the final program output.

Figure 2 — Detailed implementation flow: tokenization, infix-to-postfix conversion, and TAC generation.



8. Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a structured process for planning, designing, developing, testing, deploying, and maintaining software. The figure below outlines the seven stages followed for this compiler project, from initial planning through ongoing maintenance.

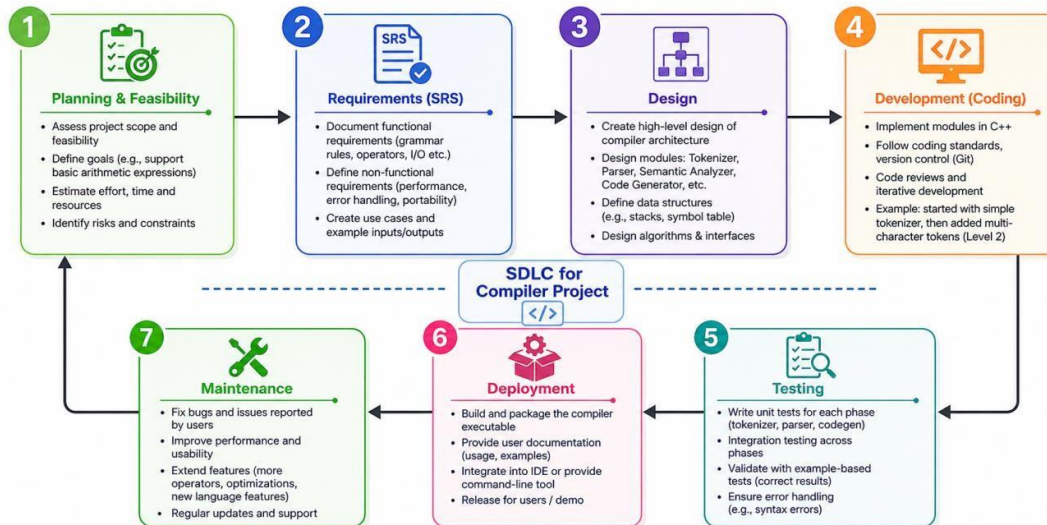


Figure 3 — SDLC stages applied to the compiler project, from planning through maintenance.

1. Planning & Feasibility

- Assess project scope and feasibility
- Define goals (e.g., support basic arithmetic expressions)
- Estimate effort, time, and resources
- Identify risks and constraints

2. Requirements (SRS)

- Document functional requirements (grammar rules, supported operators) and non-functional needs (performance, error handling, portability)
- Create use cases and example inputs/outputs

3. Design

- Create a high-level design of the compiler architecture
- Design modules: tokenizer, parser, semantic analyzer, code generator
- Define data structures (e.g., stacks, symbol table)
- Design algorithms and module interfaces

4. Development (Coding)

- Implement the modules in C++
- Follow coding standards and use version control (Git)
- Conduct code reviews and iterative development

- Example: started with a simple tokenizer, then added multi-character token support (Level 2)

5. Testing

- Write unit tests for each phase (tokenizer, parser, codegen)
- Perform integration testing across phases
- Validate with example-based tests and known correct results
- Ensure error handling for invalid input (e.g., syntax errors)

6. Deployment

- Build and package the compiler executable
- Provide user documentation, including usage and examples
- Integrate into an IDE or provide a command-line tool
- Release for users or demonstration

7. Maintenance

- Fix bugs and issues reported by users
- Improve performance and usability
- Extend features (more operators, optimizations, new language features)
- Provide regular updates and ongoing support

Adhering to SDLC best practices provides a clear roadmap and consistent quality control. By following each phase systematically, this project ensures the compiler is well-designed, thoroughly tested, and maintainable over time.

9. Conclusion

In summary, this compiler processes arithmetic expressions through a clear, staged pipeline. The **tokenizer** performs lexical analysis, the **infix-to-postfix parser** performs syntax analysis, and the **TAC generator** produces an intermediate representation suitable for further processing. Each phase has been implemented and tested in accordance with sound software engineering principles.

Moreover, integrating SDLC practices — from planning through maintenance — has guided the organization and quality of the project throughout. The result is a modular compiler prototype that illustrates the core concepts of compiler construction alongside a disciplined development lifecycle.